

You can see that the code passed to `cProfile.run()` took 28.9 seconds to complete. Pay attention to the functions with the highest total times; in this case, Python's built-in `pow()` function takes up 28.617 seconds. That's nearly the entire code's runtime! We can't change this code (it's part of Python), but perhaps we could change our code to rely on it less.

This isn't possible in this case, because `rsaCipher.py` is already fairly optimized. Even so, profiling this code has provided us insight that `pow()` is the main bottleneck. So there's little sense in trying to improve, say, the `readKeyFile()` function (which takes so little time to run that `cProfile` reports its runtime as 0).

This idea is captured by *Amdahl's Law*, a formula that calculates how much the overall program speeds up given an improvement to one of its components. The formula is $speed\text{-}up\ of\ whole\ task = 1 / ((1 - p) + (p / s))$ where s is the speed-up made to a component and p is the portion of that component of the overall program. So if you double the speed of a component that makes up 90 percent of the program's total runtime, you'll get $1 / ((1 - 0.9) + (0.9 / 2)) = 1.818$, or an 82 percent speed-up of the overall program. This is better than, say, tripling the speed of a component that only makes up 25 percent of the total runtime, which would only be a $1 / ((1 - 0.25) + (0.25 / 2)) = 1.143$, or 14 percent overall speed-up. You don't need to memorize the formula. Just remember that doubling the speed of your code's slow or lengthy parts is more productive than doubling the speed of an already quick or short part. This is common sense: a 10 percent price discount on an expensive house is better than a 10 percent discount on a cheap pair of shoes.

Big O Algorithm Analysis

Big O is a form of algorithm analysis that describes how code will scale. It classifies the code into one of several orders that describes, in general terms, how much longer the code's runtime will take as the amount of work it has to do increases. Python developer Ned Batchelder describes big O as an analysis of "how code slows as data grows," which is also the title of his informative PyCon 2018 talk, which is available at <https://youtu.be/duvZ-2UK0fc/>.